Practical Jev AI Engineering

Command Reference: every command used in the course, from first clone to production

Run the commands **in order**, module by module. Each step says what the command does and what you should see. All commands are written for **PowerShell**, which is the default terminal inside **VS Code** on Windows. Where **Git Bash** (or macOS / Linux) needs something different, it is shown right below.

How to read this sheet

You see	Meaning
A shaded box	Commands. Type or paste them into the terminal, one line at a time.
# grey line	A comment explaining the command. Do not run it.
Expect:	What a successful run looks like. Results from Jev may vary slightly.
(.venv) at the start of the prompt	The project's virtual environment is active. Most commands need this.
Output / results	Never paste output back into the terminal: PowerShell will try to run it as code.
A long command split over two lines	It is ONE command, wrapped to fit the page. If your PDF viewer adds a line break when you copy it, join it back into a single line before pressing Enter.

Which terminal?

Task	PowerShell / VS Code terminal	Git Bash / macOS / Linux
Activate the environment	`.\.venv\Scripts\Activate.ps1`	`source .venv/Scripts/activate` (macOS/Linux: `source .venv/bin/activate`)
Copy a file	`Copy-Item .env.example .env`	`cp .env.example .env`
Set a variable for one command	`$env:RUN_LIVE_TESTS = "1"; pytest -m live -s`	`RUN_LIVE_TESTS=1 pytest -m live -s`
Delete a folder	`Remove-Item -Recurse -Force .venv`	`rm -rf .venv`
Python launcher	`python` (or: `py -3.12`)	`python` (macOS/Linux: `python3`)

Open the terminal in VS Code with **Ctrl +** ` (the key under Esc). It opens in the project folder automatically. Always run commands from the **project root**: the folder that contains `pyproject.toml` and `README.md`.

Module 1 Introduction, course materials & resources

No commands. Watch the demo, open the slides and the resources in `modules/01-introduction-and-resources/`, and create your API key at **console.typesafe.ai** (API Keys). Keep the key private.

Module 2 Jev fundamentals

No commands. Concepts only: state, questions, Choice, Score, Noul and confidence. Read `app/services/questions.py` as a preview.

Module 3 Setup & your first request

From an empty computer to Jev's first real answer.

1. Get the code

Download the project from GitHub and move into it.

```
cd $HOME\Desktop
git clone https://github.com/MOHAMMEDFAHD/jev-tutorial.git
cd jev-tutorial

# Alternative: clone INTO an empty folder you already opened in VS Code (note the dot)
git clone https://github.com/MOHAMMEDFAHD/jev-tutorial.git .
```

Expect: a new `jev-tutorial` folder with app, examples, tests, modules ...

2. Create and activate a virtual environment

A private Python space for this project, so its packages don't mix with anything else.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1

# Git Bash:  source .venv/Scripts/activate
# If PowerShell says 'running scripts is disabled', run this once, then activate again:
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Expect: the prompt now starts with **(.venv)**.

3. Install the project

Installs TypeSafe's SDK, FastAPI, the database and test tools, and the browser playground. Run it **after** activating, from the project root.

```
pip install -e ".[dev,ui]"

# Check the SDK is installed in THIS environment:
pip show typesafe-sdk
```

Expect: 'Successfully installed ...' and `Version: 0.7.4`.

In VS Code: **Ctrl + Shift + P**, then **Python: Select Interpreter**, then choose **.venv**. The squiggly lines under imports disappear.

4. Create your settings file and add your key

`.env.example` is a template; `.env` is your private copy. Paste your key after `TYPESAFE_API_KEY=` (no spaces, no quotes). Never commit .env and never show it on screen.

```
Copy-Item .env.example .env
notepad .env

# Check the key is being read (it is NOT printed):
python -c "from app.config import Settings; print('key found:',
Settings().typesafe_api_key is not None)"
```

Expect: `key found: True`

5. Check everything works

Runs the automated tests. No key, no internet and no cost needed.

```
pytest -q
```

Expect: `103 passed, 1 skipped`. The skipped one is the live test (Module 8).

6. Your first real request to Jev

Sends one message and two questions, then prints Jev's typed answer and the raw JSON.

```
python examples/01_first_request.py

# No key yet? Use the offline mock (answers are NOT from Jev):
python examples/01_first_request.py --offline
```

Expect: `model that answered : jev-1.13.0`, a department, probabilities, confidence, P(urgent), and '-> would go to the billing queue'.

Module 4 Classification with Jev

Our real questions: department (Choice), priority (Score) and four human-review checks (Noul).

Run the typed-decisions example

Shows Jev's native answers to all the ticket questions, then our validated decision, then a deliberately broken answer being rejected.

```
python examples/02_typed_decisions.py
```

Expect: NATIVE ANSWERS (department, priority score, the four checks), APPLICATION DECISION, and REJECTED `tampered response -> unexpected department 'legal'`.

Module 5 Probabilities & validation

What confidence really means, and how the same answers behave under different policies.

Run the probabilities example

Three messages (clear, ambiguous, vague): probability bars, Jev's confidence next to the formula re-computed in Python, the top-2 margin, and one table of three policies.

```
python examples/03_probabilities.py
```

Expect: matching 'reported confidence' and 'formula re-computed' values; at policy 0.9 only the less certain ticket goes to human_review.

Module 6 Decision policy & routing

The full pipeline on several tickets at once: Jev judges, our policy decides, the router picks a queue.

Run the routing example

Six tickets through the whole pipeline. The offline version adds ticket T-7, which simulates Jev timing out, to show that no ticket is ever lost.

```
python examples/04_automated_routing.py

# Offline mock, with the simulated timeout (answers are NOT from Jev):
python examples/04_automated_routing.py --offline
```

Expect: one row per ticket with department, priority, action, queue and reasons (for example `needs_human:legal_threat`), then 'automation coverage: 3/6'.

Module 7 FastAPI & the database

The decision engine as a real web API, with every decision saved.

1. Start the API server

Keeps running in this terminal. Open the second address in your browser for the interactive documentation (Swagger UI). Stop the server with **Ctrl + C**.

```
uvicorn app.main:app --reload

# Then open in your browser:
# http://127.0.0.1:8000/docs
```

Expect: 'Uvicorn running on http://127.0.0.1:8000'. The docs page lists 6 endpoints.

2. Try it in Swagger (click an endpoint, Try it out, Execute)

Paste these bodies into **POST /tickets**. Use new ticket IDs each time you practise.

```
{"ticket_id": "T-200", "customer_id": "C-10", "message": "I forgot my password and the
reset email never arrived."}

{"ticket_id": "T-201", "customer_id": "C-11", "message": "My account was hacked and
someone changed my email. I'm calling my lawyer and I want to speak to your manager."}

# Same ID + same message again  -> 200 and header idempotent-replay: true
# Same ID + different message   -> 409 Conflict
# Bad data, e.g. "ticket_id": "bad id with spaces" -> 422
```

Expect: 201 with department, action, queue and reasons; the second ticket goes to the escalation queue.

3. The same calls from the terminal (optional)

Open a **second** terminal (the + in VS Code's terminal panel) while the server runs.

```
# PowerShell:
Invoke-RestMethod -Method Post -Uri http://127.0.0.1:8000/tickets -ContentType
"application/json" -Body '{"ticket_id":"T-300","customer_id":"C-1","message":"Do you offer
a discount for 50 seats?"}'
Invoke-RestMethod http://127.0.0.1:8000/metrics
Invoke-RestMethod http://127.0.0.1:8000/health

# Git Bash / macOS / Linux:
curl -X POST http://127.0.0.1:8000/tickets -H "Content-Type: application/json" -d
'{"ticket_id":"T-301","customer_id":"C-1","message":"The export button gives an error."}'
curl http://127.0.0.1:8000/metrics
```

Expect: the same JSON you see in Swagger.

4. Look inside the database

Prints the saved tickets. Personal data appears as [CARD] or [EMAIL]: the originals are never stored.

```
python -c "import sqlite3; [print(r) for r in
sqlite3.connect('tickets.db').execute('select ticket_id, department, action, reasons,
sanitized_message from tickets')]"

# Start again with an empty database (stop the server first):
Remove-Item tickets.db
```

Expect: one line per ticket, e.g. ('T-200', 'account', 'auto_route', '["all_checks_passed"]', '...').

Module 8 Testing & evaluation

Prove the system works, including when things fail, and measure decision quality.

1. Run the tests

Every test runs the real TypeSafe SDK against a faked network: no key, no internet, no cost.

```
pytest -q

# One line per test, with its name:
pytest -v

# Only the slide-correction tests:
pytest tests/test_slide_claims.py -v
```

Expect: dots for passed tests, an 's' for the skipped live test, then '103 passed, 1 skipped'.

2. The live test (calls the real Jev, needs your key)

Sends one real ticket and checks our app and Jev still understand each other.

```
$env:RUN_LIVE_TESTS = "1"; pytest -m live -s

# Git Bash / macOS / Linux:
RUN_LIVE_TESTS=1 pytest -m live -s
```

Expect: `LIVE: jev-1.13.0 -> billing (1.00), high, auto_route, ... ms` and '1 passed'.

3. Evaluate decision quality

Runs 40 labelled tickets and reports accuracy, a confusion matrix, automation coverage, accuracy on auto-routed tickets, calibration and a threshold sweep. Reports are saved in `evaluation/results/`.

```
python -m evaluation.evaluate

# Pipeline check only, NOT a measurement of Jev:
python -m evaluation.evaluate --backend mock
```

Expect: a report starting with the backend and model, ending with the threshold sweep table.

4. Code quality (formatter and linter)

Checks the code style the project uses.

```
ruff format --check .
ruff check .
```

Expect: '... files already formatted' and 'All checks passed!'.

Module 9 Production & next steps

The slides' business scenarios, the browser playground, and packaging.

1. Business scenarios from the slides

Email priority, suspicious login, invoice review and feedback, built correctly: numbers and dates are computed in Python, Jev judges the rest.

```
python examples/05_business_patterns.py
```

Expect: four scenarios, each ending with '-> ACTION (decided by our code)'.

2. The Jev playground (browser)

Type any customer message and see Jev's decision instantly. Moving a policy slider re-checks the last answer without calling Jev again. Stop it with Ctrl + C.

```
streamlit run ui/jev_playground.py

# Opens http://localhost:8501
# First run only: if it asks for an email, just press Enter.
```

Expect: a green box (routed automatically) or an orange box (held for a person) with the reasons.

3. Docker (optional, needs Docker Desktop)

Packages the API as a container.

```
docker build -t practical-jev-ai .
docker run --rm -p 8000:8000 --env-file .env -v jevdata:/data practical-jev-ai
```

Expect: the API at http://127.0.0.1:8000/docs, now running inside Docker.

Every new session

You only need to set things up once. Each time you come back:

```
cd C:\Users\<you>\Desktop\jev-tutorial
.\.venv\Scripts\Activate.ps1

# Then whatever you want to run, for example:
streamlit run ui/jev_playground.py
uvicorn app.main:app --reload

# Moving to a different project? Leave this environment first:
deactivate
```

Tip: to get the same Jev answers in every session (for example while recording), pin the model in .env: `JEV_MODEL=jev-1.13.0`

Saving your work with Git

```
git status
git add -A
git commit -m "Describe your change"
git push

# If GitHub has changes you don't have yet:
git pull --rebase
git push
```

The 'LF will be replaced by CRLF' warnings on Windows are harmless.

Starting over (practice runs)

```
# Remove the environment and the database, then repeat Module 3 from step 2:
deactivate
Remove-Item -Recurse -Force .venv
Remove-Item tickets.db

# Remove a whole practice copy (close VS Code first):
cd $HOME\Desktop
Remove-Item -Recurse -Force $HOME\Desktop\jev-practice
```

Troubleshooting

Every problem below happened during the making of this course.

Problem you see	Cause and fix
running scripts is disabled / Activate.ps1 cannot be loaded	Run once: Set-ExecutionPolicy -Scope CurrentUser RemoteSigned, then activate again.
ModuleNotFoundError: No module named 'pydantic' (or any package)	The project isn't installed in the active environment. Activate .venv, then run pip install -e ".[dev,ui]". Don't install packages one by one.
neither 'setup.py' nor 'pyproject.toml' found	You're not in the project root. Run dir pyproject.toml, then cd into the folder that has it.
Cannot find path '...\system32\.env.example'	The terminal is in the wrong folder. cd to the project first, or use VS Code's terminal.
Client setup failed: No API key was provided	Add TYPESAFE_API_KEY=... to .env, or use --offline. Check with the 'key found' command in Module 3.
401: the API key was rejected	The key is wrong or expired. Create a new one at console.typesafe.ai.
error parsing value ... / Input should be ... input_value=''	Your .env has empty or broken values. Copy a fresh .env.example to .env and add your key again.
PermissionError: Access is denied ... pytest-of-User	Folders were created by an Administrator terminal. Delete them, or run: pytest -q --basetemp="$HOME\pytest-tmp" -p no:cacheprovider. Use one kind of terminal.
93 passed, 2 skipped instead of 103 passed, 1 skipped	Streamlit isn't installed (or the wrong .venv is active). Run pip install -e ".[dev,ui]" in the right environment.
Unexpected token ... in expression	You pasted output (or a prompt like '(.venv) PS C:\...>') as a command. Paste only the commands.
!pip is not recognized	That is notebook syntax. In the terminal just use pip.
git push rejected (fetch first)	GitHub has newer commits: git pull --rebase, then git push.
Results differ a little between runs	Normal for an AI model. Pin JEV_MODEL=jev-1.13.0 in .env for consistency.